CLOUD ENGINEERING PORTFOLIO

AWS 서버리스 자가복구 & 관측성 플랫폼

장애가 나면 스스로 알아채고 마지막 정상 버전으로 되돌린다. 어떤 경우에도 예산을 넘지 않게 만든 서버리스 API(서버를 직접 관리하지 않는 API)다.

GitHub github.com/dodongwon33/aws-serverless-selfhealing · Terraform(코드로 인프라를 만드는 도구) · GitHub Actions(자동 배포 도구) · Python · 서울 리전

2단	≈ $0.6	31건	0개	59개
자동 롤백 (카나리 + 런타임)	최악 월 비용 (무료 티어 미적용)	초안 설계 결함 발견 · 수정	장기 Access Key (OIDC)	Terraform 리소스 (모듈 9개)

문제 정의

개인 서비스는 장애를 24시간 지켜볼 사람이 없다. 실수 한 번이면 큰일이 난다. 트래픽이 폭주하면 예상 못 한 큰 요금(청구서 폭탄)이 나오고, 잘못 배포하면 장애가 오래간다. 그래서 목표를 세 가지로 잡았다.

- **보이게 한다** — 로그·메트릭(수치 기록)·트레이스(요청 경로 추적)로 "무엇이, 어느 버전에서, 왜" 실패했는지 찾는다.
- **스스로 되돌린다** — 사람 없이 몇 분 안에 마지막 정상 버전으로 복구한다. 같은 결함이 다시 배포되지 않게 막는다.
- **예산을 넘지 않는다** — "무료 티어라 괜찮다"로 끝내지 않는다. 공격이나 실수가 있어도 비용 상한이 지켜지도록 구조로 보장했다.

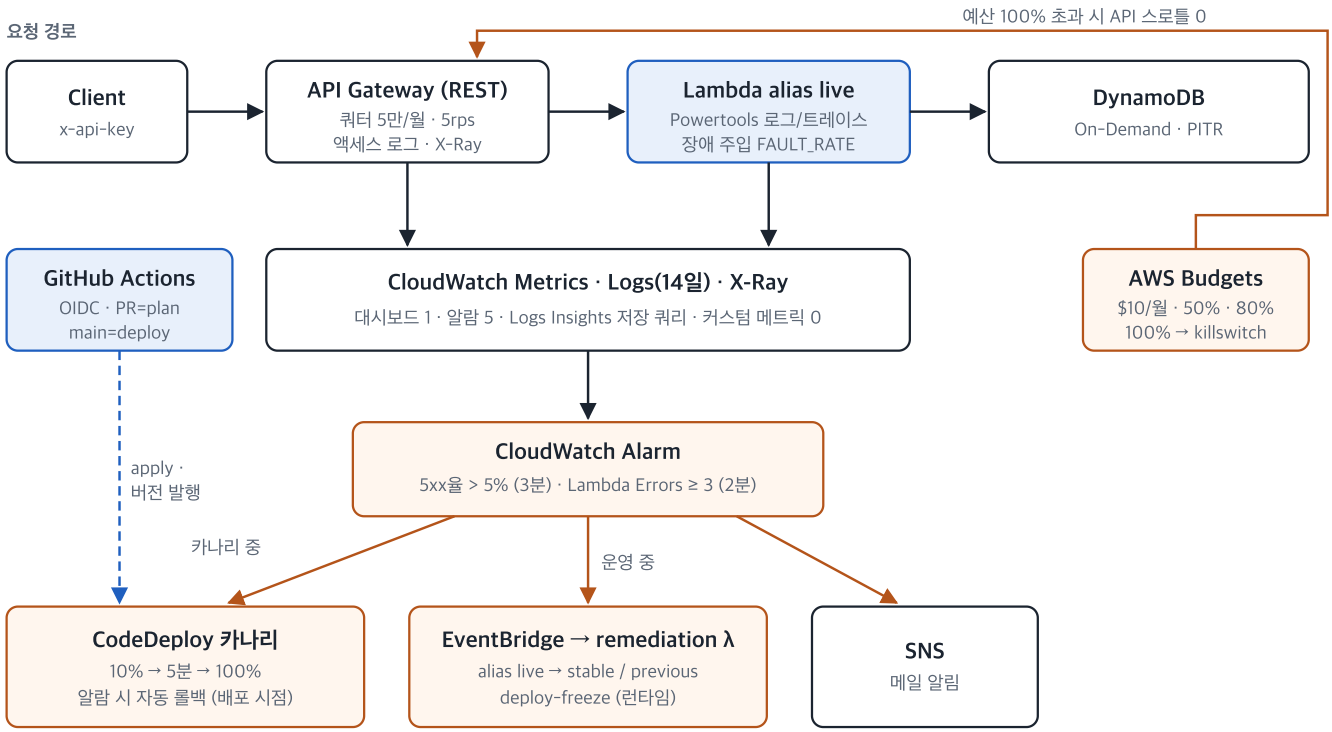
핵심 설계 결정

결정	이유
REST API (HTTP API 아님)	X-Ray(요청 추적 서비스)와 사용량 계획(월 쿼터)은 REST API만 지원한다. 쿼터 상한 기준 요금 차이는 월 $0.1 미만이다.
API → Lambda **alias**(함수 버전을 가리키는 이름표) 연결	API가 최신본(`$LATEST`)을 부르면 롤백해도 실제 트래픽에 반영되지 않는다. alias 하나로 카나리와 롤백을 모두 조절한다.
CodeDeploy 카나리(새 버전에 트래픽 10%만 먼저 보내 보는 배포 방식) + 직접 만든 remediation(복구) 함수	배포하는 순간은 관리형 서비스(CodeDeploy)에 맡긴다. 배포 이후 문제는 직접 만든 함수가 맡는다. 카나리를 건너뛴 핫픽스나 운영 중 드러난 결함이 여기에 해당한다. 둘을 합쳐 빈틈이 없게 했다.
코드 소유권 분리	Terraform은 함수 설정만, CI는 코드와 버전만 맡는다. 두 도구가 같은 리소스를 서로 덮어쓰는 문제(드리프트)를 없앴다.
쿼터 = 비용 상한	스로틀(초당 요청 제한)은 속도만 막고 총량은 못 막는다. 초당 10건이면 한 달 2,592만 건, 약 $90이다. 그래서 월 5만 건 쿼터를 걸어 API 비용 상한을 약 $0.2로 묶었다.
권한 경계(Permissions Boundary, 권한의 최대 한도선)	배포용 역할이 "관리자 역할을 만들어 그 권한을 가로채는" 길(권한 상승)을 구조적으로 막는다.

기술 스택

API Gateway(외부 요청을 받아 Lambda로 넘기는 입구) · Lambda(함수 실행) · DynamoDB(NoSQL DB) · CloudWatch(Logs/Metrics/Alarms/Dashboard/Logs Insights, 로그·감시) · X-Ray · EventBridge(이벤트 전달) · SNS(알림) · CodeDeploy(배포) · SSM Parameter Store(설정값 저장) · AWS Budgets(예산) · IAM OIDC(GitHub 인증 연동) · Terraform 1.10+ (AWS provider 6.x) · Python 3.12 · Powertools for AWS Lambda · GitHub Actions · pytest / moto(테스트 도구)

아키텍처



상태 (SSM 표준 파라미터, 무료)

stable-version: 카나리+스모크 통과 버전 · previous-version: 직전 stable · deploy-freeze: 롤백/킬스위치 후 CI 배포 차단
CI가 배포 성공 시 previous←stable, stable←new 로 승격 · remediation이 롤백 시 stable←대상, previous←none (플래핑 방지)
쉽게: 배포가 성공하면 기존 stable은 previous로 밀려나고 새 버전이 stable이 된다

용어: 5xx = 서버 쪽 오류 응답. 롤백 = 코드를 되돌리는 게 아니라 alias가 가리키는 버전 번호를 이전 정상 버전으로 바꾸는 것.

컴포넌트

구성	설정	비고
API Gateway	`GET /health` · `POST /items` · `GET /items/{id}`. API 키 필수. 스로틀은 초당 5건, 순간 최대 10건(버스트).	실행 로그·상세 메트릭은 끔 (비용)
Lambda (API)	Python 3.12, 메모리 256MB, Powertools 레이어(버전 고정), X-Ray Active, 요청 본문 10KB 제한	설정은 Terraform, 코드는 CI
Lambda (보조)	remediation(알람 → 롤백), killswitch(예산 초과 → 차단). 메모리 128MB	권한 경계 적용
DynamoDB	`items` 테이블(키는 id), On-Demand(쓴 만큼 과금), PITR(시점 복구), AWS 소유 키로 암호화	월 1원 미만
Terraform	bootstrap(state 저장소·OIDC·역할) + dev 환경, 모듈 9개, S3 네이티브 락(동시 작업 잠금)	DynamoDB 락 테이블 불필요

자가복구 — 2단 자동 롤백

롤백 기회는 두 번이다. **배포하는 순간**에는 CodeDeploy가 트래픽 10%만 새 버전으로 보내 5분간 지켜본다. **배포한 이후**에는 알람이 EventBridge를 거쳐 직접 만든 remediation 함수를 깨운다.

런타임 remediation 의사결정

1. 필터
ALARM 상태가 아니면 무시

2. 위임
카나리 진행 중이거나 최근 20분 내 실패했으면 CodeDeploy 담당

3. 중복 방지
이미 stable로 돌아가 있고 동결도 끝났으면, 같은 알람이 또 와도 롤백 없이 알림만

4. 대상 결정
stable / previous / 수동

5. 실행
alias 갱신 → 상태 갱신 → 배포 동결 → 메일

용어: live = 지금 서비스 중인 버전, stable = 검증 통과한 마지막 정상 버전, previous = 그 직전 정상 버전.

상황	버전 상태	처리
카나리를 건너뛴 핫픽스가 장애	live=7, stable=5	v5로 롤백
검증을 통과한 버전이 운영 중 장애	live=stable=5, previous=4	v4로 롤백, previous 비움
되돌아갈 버전 없음 (최초 배포)	live=stable, previous=none	롤백 없이 동결 + "수동 조치 필요" 메일
두 알람(5xx율·Errors) 동시 발생	첫 실행이 이미 롤백·동결함	두 번째 실행은 `already_frozen` (이미 동결됨)

설계하며 막은 실패 모드

연쇄 롤백
CodeDeploy가 먼저 롤백했는데 알람이 늦게 오면, "진행 중 배포 없음"으로 보여 정상 버전을 또 되돌린다. 그래서 최근 20분 내 실패·중단 배포도 CodeDeploy 담당으로 본다.

플래핑(왔다 갔다 반복)
롤백 뒤 같은 알람이 또 울리면 나쁜 버전으로 돌아갈 수 있다. 그래서 롤백한 버전을 새 stable로 두고 previous는 비워 길을 끊었다.

재배포 사고
롤백 직후 다음 push가 같은 결함을 다시 배포할 수 있다. `deploy-freeze` (배포 동결)로 CI를 막고, 원인을 고친 뒤 수동으로 푼다.

저트래픽 카나리
평소 요청이 0건이라 카나리 5분 동안 알람이 판단할 데이터가 없다. 그래서 CI가 가짜 요청(합성 트래픽, 초당 2건 × 7분)을 보낸다. 알람은 분당 20건 미만이면 0%로 처리해 오탐을 막는다.

복구를 증명하는 장애 시연

Lambda 버전은 환경변수까지 저장한다. `scripts/chaos_bad_release.sh` 는 `FAULT_RATE=0.5` (요청의 절반을 일부러 실패시킴)인 버전을 만든다. 그 버전을 카나리 없이 live에 바로 올리고, 초당 3건 부하를 건다. 예상 흐름은 다음과 같다.

0분
장애 버전 live 반영, 응답의 약 50%가 502

~3분
5xx율 알람 ALARM

~3.5분
EventBridge → remediation → alias를 stable로

~4분
200 응답 복귀, 메일 수신, 배포 동결

> 구현 상태: 코드와 IaC(코드로 쓴 인프라)는 완성됐다. 단위 테스트, 정적 검증, 실제 계정 대상 `terraform plan` 까지 통과했다. 실제 배포와 장애 시연 결과(대시보드 캡처, 복구 시간 측정)는 배포 후 이 섹션에 추가할 예정이다.

CI/CD (코드 검사와 자동 배포)

워크플로	트리거	단계
`ci.yml`	Pull Request	ruff(코드 검사) · pytest(테스트) · `terraform fmt/validate` → 읽기 전용 역할로 `plan`(변경 미리보기) → PR 코멘트
`deploy.yml`	main push · 수동	테스트 → 배포 동결 확인 → `terraform apply`(실제 반영) → 버전 발행 → CodeDeploy 카나리(합성 트래픽 포함) → 스모크 테스트(health + 쓰기/읽기 왕복) → stable 승격

AWS 연결 전(저장소 변수 미등록)에는 배포 job을 자동으로 건너뛰고 테스트만 돈다. 동시에 배포가 겹치지 않도록 `concurrency`로 한 줄로 세웠다.

보안

장기 Access Key 0개

GitHub OIDC(GitHub가 "이 요청은 내 저장소가 보냈다"고 보증하면 AWS가 임시 권한을 주는 방식)로 실행할 때마다 임시 자격증명을 받는다. `sub` 값(요청자 표시)으로 역할을 나눴다.

- `repo:…:ref:refs/heads/main` → 배포 역할
- `repo:…:pull_request` → 읽기 전용 plan 역할

권한 경계로 권한 상승 차단

배포 역할은 IAM 역할을 만들 수 있어야 해서, 그냥 두면 관리자 역할을 만들어 권한을 가로챌 수 있다. 그래서 "이 한도선이 붙은 역할만 만들 수 있다"는 규칙(`iam:PermissionsBoundary` 조건)을 걸었다. 만든 역할이 아무리 강해도 한도선 밖 권한은 못 쓴다. 경계를 제거하거나 bootstrap 역할을 수정하는 것도 막았다(`/selfheal/` path 한정).

리소스 스코프(권한 범위)

Lambda·DynamoDB·SNS·SSM·CloudWatch·CodeDeploy 권한은 이름이 `selfheal-*`인 리소스(ARN)로 한정했다. API Gateway는 ARN이 랜덤 ID라 리전 단위로 한정했다. 알려진 한계다.

비밀 정보 관리

알림 메일은 GitHub Secret → `TF_VAR`로 넣고 `sensitive`로 표시해 PR plan 코멘트에 안 보이게 했다. API 키는 CI 로그에서 가린다. state 버킷은 버전관리, SSE-S3 암호화, 퍼블릭 차단, TLS 강제를 적용했다.

> SNS를 암호화하지 않은 것은 의도한 선택이다. CloudWatch 알람은 AWS 관리형 키(aws/sns)로 암호화한 토픽에 발행할 수 없다. 고객 관리형 KMS 키는 월 $1이라 예산 대비 과하다. 토픽에는 알람 메타데이터(부가 정보)만 흐른다.

비용 설계 — "무료 티어"가 아니라 "상한"

2025년 7월 프리티어 개편 이후 기존 계정의 12개월 무료 혜택(API Gateway 등)이 끝났을 수 있다. 그래서 **무료 티어가 전혀 없다고 가정**했다. 쿼터 월 5만 건을 전부 쓰는 최악의 경우로 계산했다.

항목	산정 근거	월
API Gateway REST	5만 건 × 약 $4/백만	$0.20
Lambda	요청 $0.2/백만 + 0.25GB × 0.2초 × 5만	$0.05
DynamoDB On-Demand	쓰기 약 1만 건, 저장 수 MB	$0.02
CloudWatch Logs	수집 약 0.1GB × $0.76	$0.08
X-Ray	기본 샘플링, 최대 5만 트레이스 × $5/백만	$0.25
알람 5 · 대시보드 1 · 저장 쿼리	상시 무료 한도(알람 10 · 대시보드 3) 이내	$0
S3 · SSM · EventBridge · SNS · CodeDeploy · Budgets	무료 또는 극소. Budgets는 Budget Action을 안 써서 무료	~$0.01
합계 (최악)		**≈ $0.6 (약 900원)**

다층 예산 방어 — AWS에는 하드 지출 상한(넘으면 강제로 멈추는 한도)이 없다

① 쿼터
사용량 계획 월 5만 건 = 요청 수 자체의 상한 (실질적 비용 상한)

② 조기 경보
Budgets $10(계정 전체): 실제 50% · 예측 80%에 메일

③ 킬스위치(긴급 차단)
실제 100% → SNS → Lambda → API 스로틀 0 + 배포 동결

킬스위치는 Budget Action(유료일 수 있음) 대신 SNS 알림 → Lambda로 만들어 추가 비용이 없다. Budgets 데이터는 몇 시간 늦게 갱신된다. 그래서 킬스위치는 마지막 안전망이고, 실질적인 상한은 ①이다.

의도적으로 쓰지 않은 것

제외	이유
NAT Gateway · VPC 내 Lambda	NAT 하나에 월 약 $43이다. 이 워크로드는 VPC(가상 사설망)가 필요 없다.
API GW 메서드 상세 메트릭	유료 커스텀 메트릭으로 과금된다. 스테이지 기본 메트릭이면 충분하다.
API GW 실행 로그	보존기간 무제한 로그 그룹이 자동 생성된다. 액세스 로그(14일)로 대체했다.
고객 관리형 KMS · Secrets Manager	키당 월 $1, 시크릿당 월 $0.4다. OIDC를 써서 관리할 시크릿이 없다.
CloudWatch Synthetics · Lambda Insights	실행·함수 단위로 과금된다. 합성 트래픽은 CI에서 필요할 때만 만든다.

설계 검토 — 초안의 결함 31건

초안 설계를 그대로 만들지 않았다. 코드로 옮기며 `validate`, 실제 계정 대상 `plan`, 단위 테스트로 검증했다. 그 과정에서 찾은 결함은 다음과 같다(전체 목록은 `docs/REVIEW.md`).

심각도	초안	문제 → 수정
치명	HTTP API + X-Ray로 요청이 지나간 경로를 그림으로 보기	X-Ray는 REST API만 지원 → REST 채택
치명	"별칭으로 이전 버전 롤백"	API가 `$LATEST`를 불러 롤백이 소용없음 → alias 통합 + 버전 발행
치명	롤백 대상 결정 로직 없음	stable/previous 상태 관리, 플래핑 방지
치명	카나리·알람 기반 검증	트래픽이 적어 알람 데이터 없음 → 합성 트래픽
치명	(계정 상태 미확인)	OIDC Provider가 이미 있어 apply 실패 → 재사용 옵션
치명	(구현 중 테스트로 발견)	DynamoDB Decimal(소수 타입)이 문자열로 바뀜 → 커스텀 serializer(변환 코드)
높음	"초당 10건 스로틀로 비용 방지"	총량 제한이 아님(최악 약 $90) → 쿼터 + Budgets + 킬스위치
높음	"최소권한 IAM"	IaC 역할에 권한 상승 경로가 있음 → 권한 경계
높음	자동 롤백 + CI/CD	재배포 사고, 이중·연쇄 롤백 → 배포 동결, CodeDeploy에 위임
높음	로그·메트릭 수집	상세 메트릭 과금, 실행 로그 무제한 보존 → 끔(OFF)
중간	DynamoDB 쓰로틀 자동복구	On-Demand는 용량이 자동으로 늘어나 일부러 쓰로틀을 만들기 어렵다 → 시연 불가, 제거
중간	DynamoDB state 락 테이블	deprecated(더는 쓰지 말라고 권장함) → S3 네이티브 락

검증

항목	결과
pytest + moto (API 8 · 자동복구 11 · 킬스위치 1)	20 passed
ruff lint / format	통과
terraform fmt / validate (bootstrap, dev)	통과
terraform plan — 실제 AWS 계정, 읽기 전용	bootstrap 11개 · dev 59개 리소스, 오류 없음
실제 배포 · 장애 시연	예정 (예상 비용: 시연 1회 약 $0.01)

배운 점 · 다음 단계

- **"무료"는 설계가 아니다.** 무료 티어에 기대지 말고, 쿼터처럼 구조적인 상한으로 비용을 지켜야 한다.
- **자동화끼리 서로 부딪친다.** 관리형 롤백, 직접 만든 롤백, CI가 서로를 되돌리지 않게 위임·동결 규칙을 정해야 한다.
- **설계 문서는 코드로 검증해야 한다.** 31건 중 다수는 문서만 읽어서는 안 보였다. plan과 테스트에서 드러났다.
- 다음 단계: CodeDeploy `BeforeAllowTraffic` 훅(트래픽 받기 전 점검)으로 새 버전 사전 검증, Trivy/checkov IaC 보안 스캔, Slack 알림, 운영 환경 계정 분리.